

Project Name: Multimodal Temporal Emotion Recognition on MELD

Alternative Names: Emotion Recognition Repo, Multimodal Fusion Project, Audio Video Text AI, MELD Classifier

GitHub Link: <https://github.com/brij26/multimodal-emotion-recognition>

1. Project Purpose & Overview

- **What it does:** This project is a state-of-the-art multimodal emotion recognition system that blends three distinct data streams—text, audio, and video—to classify emotional states across 7 distinct categories using the MELD dataset.
- **Why I built it:** I built this to solve the classic limitation of single-modality models. While text-only models struggle with sarcasm or tone, this tri-modal system dramatically improves classification robustness, achieving a 70 to 75% Weighted F1 score compared to a standard 58% text-only baseline.

2. Tech Stack

- **Core Framework:** PyTorch 2.0+ and Hugging Face Transformers.
- **Feature Extractors:** BERT for text processing, Wav2Vec2 for audio processing, and Vision Transformers (ViT) combined with a BiLSTM for temporal video processing.
- **Optimization & Math:** Focal Loss for class imbalance mitigation and Bootstrap Confidence Intervals for rigorous statistical evaluation.

3. Architecture Diagram & Design

- **System Flow:** Raw inputs are processed through a precomputed embedding pipeline to maximize training efficiency. Text is converted into a 768-dimensional vector using BERT's CLS token. Audio is passed through Wav2Vec2 to extract 768-dimensional mean-pooled hidden states. For video, 8 frames are uniformly sampled and fed into a Vision Transformer (ViT); these frame embeddings are sequentially processed by a Bidirectional LSTM and pooled via self-attention into a final 768-dimensional video vector. These three modalities are stacked as individual tokens to form a tensor of shape batch size by 3 tokens by 768 channels, which undergoes an 8-head Multi-Head Cross-Attention mechanism for inter-modal reasoning. The attended features are flattened and passed through an MLP classifier with Dropout to output the final emotion label.

4. Design Tradeoffs

- **Tradeoff 1 - Compute Efficiency vs. Joint Fine-Tuning (Precomputed Embeddings):** I made a conscious design choice to precompute and freeze the embeddings from BERT, Wav2Vec2, and ViT into offline files before running the fusion training. The tradeoff was losing the ability to jointly fine-tune the heavy feature extractors end-to-end. However, this dramatically reduced GPU memory consumption and training time, allowing for rapid

experimentation with different bimodal and tri-modal attention architectures on consumer-grade hardware.

- **Tradeoff 2 - Inter-Modal Representation vs. Inference Latency (Cross-Attention):** Instead of simply concatenating the text, audio, and video vectors together, I treated them as distinct tokens inside an expensive Multi-Head Cross-Attention block. The tradeoff here is a slight increase in computational latency during a forward pass. I accepted this because cross-attention allows the model to dynamically weight interactions between modalities—like using the audio signal to ground ambiguous text—yielding a significant F1 score boost over primitive concatenation.

5. What I Would Do Differently

- **Hindsight adjustments:** If I were to build on this framework, I would implement parameter-efficient fine-tuning techniques like LoRA (Low-Rank Adaptation) directly on top of the foundation model backbones instead of using static precomputed embeddings. This would allow the encoders to adapt specifically to emotional nuances in voice and facial expressions without blowing up the compute budget. Additionally, I would export the final fusion layer and the compact encoders via TorchScript or ONNX to build a low-latency, real-time inference pipeline suitable for live video streams.

6. Notable Implementation Details

- **Key Challenge Solved (Audio Silence Masking):** Real-world multimedia datasets contain video clips where a speaker is silent. Passing empty audio signals through Wav2Vec2 injects noise into the cross-attention layer. To solve this, I built an automated amplitude threshold detector. If a segment is flagged as silent, the system bypasses extraction and injects a zeroed-out neutral vector, preventing corrupted audio from degrading the tri-modal classifier's performance.
- **Statistical Reliability (Bootstrap Validation):** To ensure the F1 scores were genuinely stable and not a product of overfitting the test set split, I implemented a custom bootstrap-based confidence interval script. Running 500 resampled iterations allowed me to mathematically guarantee a true 95% confidence interval for our tri-modal performance metrics.